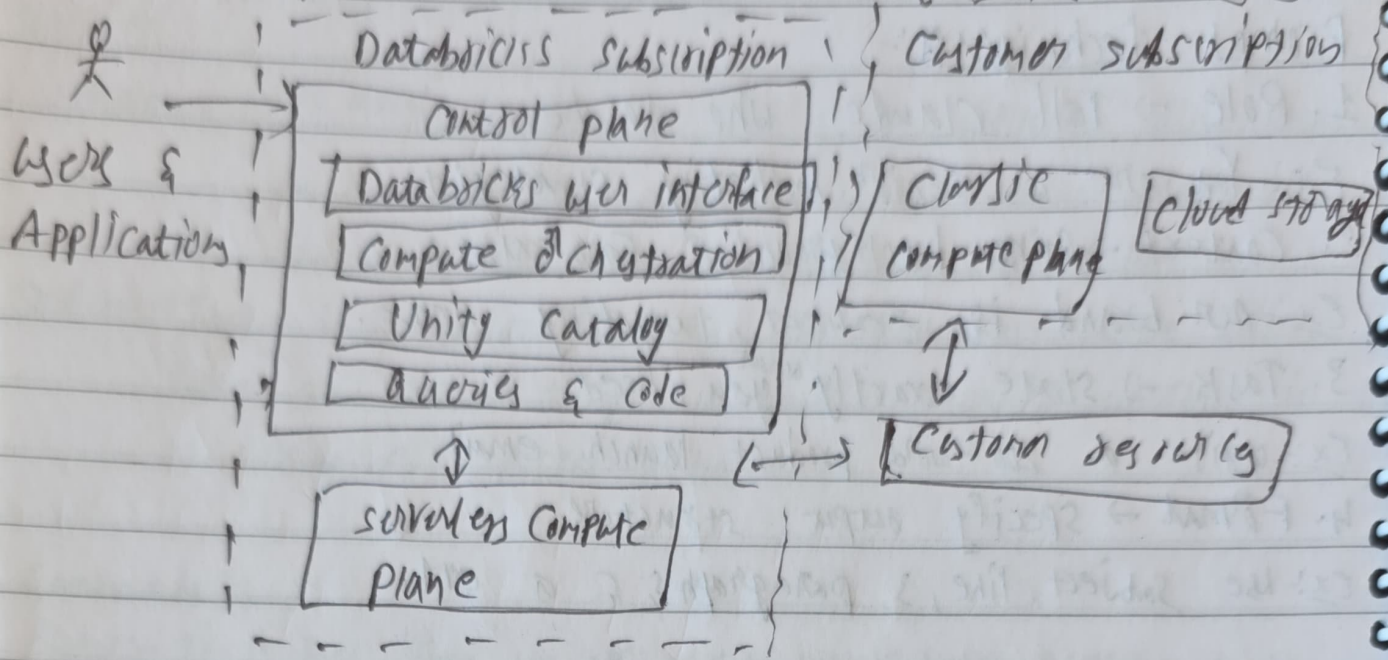


Databricks (ADEC) (by Udemy)

Architecture



Magic Commands:-

1. `%python`, `%sql`, `%scala`, `%r` → Switch to different languages
2. `%md`
3. `%fs` → run file system commands
4. `%sh` → run shell commands
5. `%pip` → Install python libraries.
6. `%runrun` → Include/import another notebook into the current notebook

Configure Access to Cloud storage:- (V 29, 30)

1. Create Access connection
 2. Create Azure Data lake storage.
 3. Assign storage Blob data Contributor role
 4. Create storage credential
 5. Create external location.
- } Azure.
} Databricks

Some commands:-

- * `show catalog;` → SQL
- * `select current_catalog();` → SQL (To know which one we are using)
- * `Create catalog if not exists Name`
managed location 'path'
comment 'comment'; → SQL (To create catalog)
- * `Use catalog catalog-name;`
- * `Create schema if not exists schema-name`
managed location 'Path';
- * `show Schemas;`
- * To create volume we have to be in schema.
- * `Create External Volume if not exists volume-name`
Location 'path';

SQL

- * `select * from <file-format>. <file-path>.`
ex: `select * from JSON. 'path'` → for single file
`select * from JSON. 'path'*` → for multiple files.
- * `select -metadata.file-path as file-path, * from JSON. 'path'.`
This command is used to get file path for each record.

View:-

- * `Create or replace view view-name`
AS
`select * from JSON. 'path'`
- * It holds the SQL query whenever we are executing the view that query will execute & shows the output. It means view stores the query.

Temporary view:-

- * This table is available for until spark session ends.
- * It stores data temporarily.

Create & replace Temporary view view-name
AS

```
select * from json.`path`.
```

Global temporary view:-

- * This table is available for until spark cluster ends.

Create & replace Global Temporary view view-name
AS

```
select * from json.`path`.
```

- * Created Global temporary views present in global-temp schema.

CSV file with header & other delimiter in SQL:-

```
select * from read-file ('path', format => 'csv',  
                        delimiter => '|', header => true);
```

- * There is a documentation of "read-files" It is a function.

Create external table:-

Create table if not exists table-name (Column Datatype, ...)

using CSV options

```
header = "true";
```

```
delimiter = "|"
```

```
)
```

Location 'path'.

- * Path we have to give external.

- * If file added & deleted we have to refresh the table to update meta data.

Refresh table table-name;

* If we drop table also it is present in ^{external location} ~~catalog~~ but not in databases this is the difference for external & managed table

Q) How to run SQL Commands using pyspark:-

SPARK.SQL (command)

Ex:- df = spark.sql (command)
display(df).

How to read file in ~~spark~~ Spark Dataframe Reader API:-

1. df = spark.read.format('json').load('path')
display(df)

2. df = spark.read.json('path')
display(df).

SPARK table Function

df = spark.table ('Table-name')
display(df)

Some Commands:-

* Select Count If (column is null), .. from table:

To remove duplicates & get latest data only:-

```
select col1,  
max (col2),  
max (col3), ...
```

from table-name

where col2 is not null

group by col1,
order by col1.

Changing data type in SQL:

select col1 (col1 as datatype), col2, col3 (col3 as datatype) from tb.

SPLIT:

Ex: select split(col1, ',') [i] as Alias from tb

- * It returns value in array so we have to use indexes to extract data.

PIVOT: (I have to see more on it)

It is used to ~~convert~~ ^{convert} rows to columns.

Array distinct:

- * It removes duplicates in array.
- * It mainly used in json format files.

Ex: select array_distinct(col) from tb.

Explode:

Ex: select explode(array_distinct(col)) from tb.

- * It is used to divide array into individual columns.

User defined functions (UDF):

Ex: create or replace function catalog-name.schema-name.

udf-name (parameters datatype)

returns return-type

return expression;

- * If we are not specifying catalog & schema where it have to store it will store default schema.

Ex: Create a sql function given below default.get-fullname
(first-name STRING, surname STRING)

Returns string

return CONCAT (initcap (first-name), ' ', initcap (surname));

Higher order functions:-

- * Higher order functions are functions that operates on complex data types such as arrays & maps.
- * They allow you to pass functions as arguments (such as lambda expressions) apply transformations & return arrays & maps.
- * They are extremely useful for manipulating arrays without exploding them.

Commonly used Higher order Array Functions:-

* Transform

* Filter

* Exists

* Aggregate

Syntax:- function-name (array-column, lambda-expression).

Ex for transform:-

select col, transform(col, x → upper(x)) as upper-items from TB;

Ex for filter:-

select col, filter(col, x → x like 'x-smartx.') as smart from TB;

Ex for exists:-

select col, exists(col, x → x = 'monitor') as monitor from TB;

Map Functions :-

A map is a collection of key-value pairs, like a dictionary.

Commonly used Higher order Map Functions :-

* Transform - values

* Transform - Keys

* Map - filter.

Syntax :-

function - name (map, column, lambda-expression).

Ex for transform - Keys :-

```
select col, transform-keys (item-price, (item, price) -> upper(item))  
as item-upper from TB;
```

Ex for transform - values :-

```
select col, transform-value (item-price, (item, price) -> round(price*1.2))  
as price-with-tax from TB;
```

Ex for map - filter :-

```
select col, map-filter (item-price, (item, price) -> price > 500) as  
premium-items from TB;
```